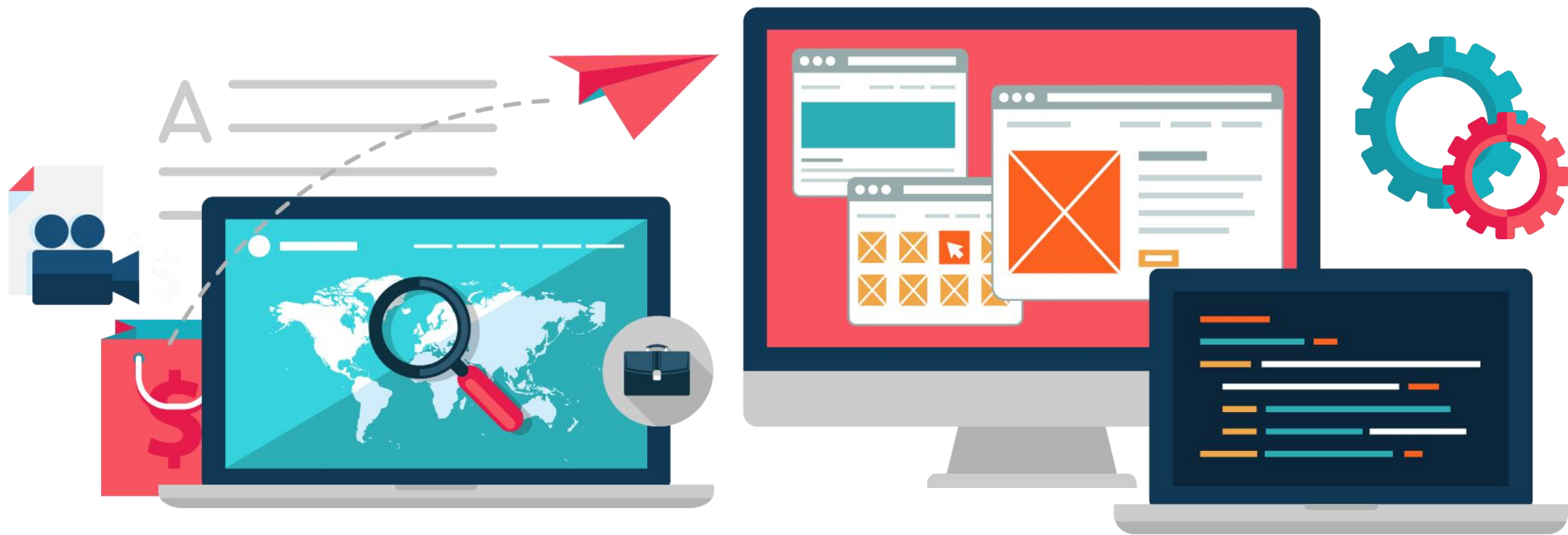


# 面向对象程序设计

Object Oriented Programming





# 成员函数



01

成员函数的实现

02

this指针

03

外联实现和内联实现

04

访问控制

05

常成员函数

06

实例变(常)量/实例方法/类变量/类方法

07

Card类的定义和实现





```
class Student {  
public:  
    Student(int sc,int st) ;  
    void study(int hours);  
    void exercise( );  
private:  
    int otherFunc( );  
    ...  
private:  
    int score;  
    int strength;  
    ...  
};
```

行为的函数表示:

- ◆ 一般成员函数
- ◆ 常成员函数
- ◆ 重载的成员函数
- ◆ 静态成员函数/类方法
- ◆ 构造、析构、拷贝构造、赋值函数
- ◆ 转换函数

数据的表示:

- ◆ 实例变量
- ◆ 类变量

```
class Student {  
public:  
    Student(int sc,int st)  
    { score = sc; strength =st; }  
  
    void study(int hours);  
    void exercise();  
private:  
    int otherFunc();  
    ...  
private:  
    int score;  
    int strength;  
    ...  
};
```

```
void Student::study(int hours) {  
    score += hours;  
    strength -= 2;  
    OtherFunc( );  
}  
void Student:: exercise( ) {  
    strength += 3;  
}
```

```
int main( ) {  
    Student s1(0,100), s2(50,80);  
    s1.study(4); s1.exercise( );  
    s2.study(2); s2.exercise( );  
    return 0;  
}
```

- ✓ 每个成员函数的实现只有一个，类的任何对象调用的都是这一个实现。
- ✓ 成员函数里没有出现任何一个具体的对象名，例如s1或s2，那它是怎么知道该去修改哪个对象的数据成员呢？
- ✓ C++编译器默默地为每个非静态成员函数增加了一个隐藏的参数——this指针。

//student.h 文件

```
class Student {  
public:  
    Student(int sc,int st)  
    { score = sc; strength =st; }  
  
    void study(int hours);  
    void exercise();  
private:  
    int otherFunc();  
  
    ...  
private:  
    int score;  
    int strength;  
    ...  
};
```

//student.cpp

```
void Student::study(int hours) {  
    //void Student::study(Student * const this, int hours)  
    this->score += hours;  
    this->strength -= 2;  
    this->OtherFunc( );  
}  
void Student:: exercise( ) {  
    //void Student:: exercise(Student* const this)  
    this->strength += 3;  
}
```

//main.cpp文件

```
int main( ) {  
    Student s1(0,100), s2(50,80);  
    s1.study(4); s1.exercise( );  
    s2.study(2); s2.exercise( );  
    return 0;  
}
```

- 是一个关键字，也是一个保留字；
- 是**非静态**成员函数**隐含的第一个形参变量**；
- 其类型：T \* **const** this，是指向当前对象的指针常量

```
int main( ) {  
    Student s1(0,100),s2(50,80);  
    s1.study(4); s1.exercise( );  
    s2.study(2); s2.exercise( );  
    return 0;  
}
```

## 内部调用:

```
Student::study(&s1,4); //this指向s1, 修改s1的成员  
Student::exercise(&s1); //this指向s1, 修改s1的成员  
Student::study(&s2,2); //this指向s2, 修改s2的成员  
Student::exercise(&s2); //this指向s2, 修改s2的成员
```

- this的作用域和生存期是在非静态成员函数的{ } 内
- 谁调用成员函数，this就指向谁，this指针永远指向当前对象
- 局部变量名与数据成员名冲突时，若要访问同名的数据成员，需显式使用this

```
//student.h
class Student {
public:
    Student(int sc,int st)
    { score = sc; strength =st; }

    void study(int hours);
    void exercise();
    void help(Student & st, int score)
    void help(Student & st)
private:
    int otherFunc();
    ...
private:
    int score;
    int strength;
    ...
};
```

```
//student.cpp

void Student::help(Student &st, int score)
{
    score += 5;
    this->score +=5;
}

void Student::help(Student & st)
{
    score += 5;
    st.score +=5;
}
```

问题1、2个help函数中的score += 5含义一样吗？  
问题2、要对当前对象的score加5应该怎么写？  
问题3、要对st对象的score加5应该怎么写？

## this指针必须显式使用的场景：返回对象自身

```
class Ultraman {
public:
    Ultraman() { happiness=0; }
    int getHappiness() const
    { return happiness; }
    Ultraman& fight(Monster&);
    bool fight(Boss &);
private:
    int happiness;
};
```

```
bool Ultraman::fight(Boss & b)
{
    bool win=...;
    if (win) happiness+=100;
    else    this->happiness =0;
    return win;
}
Ultraman& Ultraman::fight(Monster& m)
{
    ...
    happiness +=5;
    return *this;
}
```

```
int main() {
    Ultraman super; Monster m1; Monster m2; Boss b;
    if( super.fight(b) ) cout<< "胜利了: " <<super.getHappiness()<<endl;
    else  cout<< "失败了: " <<endl;
    cout<<super.fight(m1).fight(m2).getHappiness()<<endl;
}    //从调用者角度考虑接口契约：fight函数应返回当前对象的引用，而非其他类型
```



## this指针必须显式使用的场景：返回对象自身

```
class Ultraman {
public:
    Ultraman() { happiness=0; }
    int getHappiness() const
    { return happiness; }
    Ultraman* fight(Monster&);
    bool fight(Boss &);
private:
    int happiness;
};
```

```
bool Ultraman::fight(Boss & b)
{
    bool win=...;
    if (win) happiness+=100;
    else    this->happiness =0;
    return win;
}
Ultraman* Ultraman::fight(Monster& m)
{
    ...
    happiness +=5;
    return this;
}
```

```
int main() {
    Ultraman super; Monster m1; Monster m2; Boss b;
    if( super.fight(b) ) cout<< "胜利了: " <<super.getHappiness()<<endl;
    else  cout<< "失败了: " <<endl;
    cout<<super.fight(m1)->fight(m2)->getHappiness()<<endl;
}
```

外联实现是指将类的成员函数的定义（函数体）放在类定义之外、通常位于独立的 .cpp 实现文件中，而不是写在类定义内部或头文件中。

```
//car.h
class Person;
class City;

class Car {
public:
    Car( Person& owner);
    int Move( int miles );
    int Move(const City& destCity);
    void Brake( );
    //...other
};
```

```
// car.cpp
#include "car.h"
#include "person.h"
#include "city.h"

Car::Car( Person& owner) {
    owner.somefunc1( );
}
int Car::Move(const City& destCity) {
    destCity.somefunc2( );
}
void Car::Brake( ) {
}
//...other
```

- 1.注意前置声明的条件和包含头文件的时机;
- 2.前置声明降低编译依赖传递，修改头文件时，只影响直接包含它的 .cpp，不影响间接使用者



- ◆ 在类体外定义的成员函数具有外部链接性，因此，通常将类体外定义的成员函数放置在源文件中，而将类定义放置在头文件中
- ◆ 分文件放置的好处是：
  1. 类的实现文件通常较大，分开便于阅读、管理和维护；
  2. 类的实现者可以只公开包含类定义的头文件，而不用公开包含类成员函数定义的实现文件；
  3. 将类定义放在头文件中，需要时不必再定义，只需包含对应头文件即可；
  4. 便于团队成员分工协作开发



```
//t.h
#ifndef TH
#define TH
#include <iostream>
using namespace std;
class T {
public:
    void f( ) {
        cout<<"void f( ) { }"<<endl;
    }
    void g( ) {
        cout<<"inline void g( ) { }"<<endl;
    }
    void h( );
    void k( );
};
#endif
```

//或者使用inline关键字

```
inline void T::g( )
{
    cout<<"inline void g() { }"<<endl;
}
```

内联的作用：

**建议**编译器在调用处直接展开函数代码，编译器拥有对任何未标记为inline的函数使用内联替换的自由，和对任何标记为inline的函数生成函数调用的自由。



## inline关键字:

- 内联函数的定义必须在访问它的编译单元中可达
- 在程序中可有多于一次定义，只要每个定义都出现在不同编译单元中且所有定义等同即可。例如，内联函数可定义于被多个源文件所 `#include` 的头文件中。
- 对于函数的意义已经变为“容许多次定义”而不是“优先内联”

```
//t.h
#ifndef TH
#define TH
#include <iostream>
using namespace std;
class T {
public:
    void f()
        { cout<<"void f() {}"<<endl; }

    inline void g()
        { cout<<"inline void g() {}"<<endl; }

    inline void h(); //声明优先, 最佳实践
    void k();
};
inline void T::h() { } // 隐式继承 inline 属性
inline void T::k() { } // 显式强制内联, 覆盖类内普通声明
#endif
```



| 场景                         | 是否内联    | 推荐做法           |
|----------------------------|---------|----------------|
| 在类定义内部直接定义                 | 是（隐式内联） | 简洁直观，推荐用于简单函数  |
| 类内声明无 inline，类外定义无 inline  | 否（普通函数） | 用于可能被多处调用的复杂函数 |
| 类内声明有 inline，类外定义省略 inline | 是       | 推荐，声明处清晰表达意图   |
| 类内声明有 inline，类外定义有 inline  | 是       | 冗余，不推荐         |
| 类内声明无 inline，类外定义有 inline  | 是       | 可行，但意图不直观，不推荐  |

- ◆ 内联函数定义应放在头文件中
- ◆ 如果函数声明为inline，那么在所有出现该函数声明的编译单元中，都必须被声明为inline



## 使用一般内联的不足(例)

```
//a.h
#ifndef AH
#define AH

class B;
class A {
public:
    void af( B& b) { b.bf ( ); }
    void ag( ) { }
};

#endif
```

```
//a.cpp
#include "a.h"
#include "b.h"
...
```

```
//b.h
#ifndef BH
#define BH

class A;
class B {
public:
    void bf( ) { pa->ag ( ); }
private:
    A * pa;
};

#endif
```

```
//b.cpp
#include "a.h"
#include "b.h"
...
```

```
//a.h
#ifndef AH
#define AH

class B;
class A {
public:
    void af( B& b); { b.bf(); }
    void ag( ) { }
};

#endif
```

```
//a.cpp
#include "a.h"
#include "b.h"
void A::af( B& b)
{ b.bf ( ); }
...
```

```
//b.h
#ifndef BH
#define BH

class A;
class B {
public:
    void bf( ); { pa->ag(); }
private:
    A * pa;
};

#endif
```

```
//b.cpp
#include "a.h"
#include "b.h"
void B::bf( )
{ pa->ag ( ); }
...
```

- ✓ 当两个类相互依赖且内联实现时，互相包含头文件必然导致一方无法获得完整类型定义，形成死锁。
- ✓ 当两个类相互依赖，且互相作为指针或引用型数据成员或函数参数时，必须使用前向声明配合外联实现来打破头文件循环包含。
- ✓ 如果互相作为值类型数据成员，则逻辑上无法实现（因为一个对象的大小会依赖另一个对象的大小，形成无限递归），此时必须改用指针/引用。



- public: 任何类都可访问
- private: 本类(**注意: 不是本对象**)或友元可以访问
- protected: 对本类相当于private,对派生类以后再讲解
- 不可访问: 以后再讲解

```
// 例
class Course {
public:
    int getDifficulty( )
    { return difficulty;}
private:
    int preiodCount( );
    int courseId;
    int difficulty;
};
```

```
class Student {
public:
    Student(int sc,int st);
    void study(Course & course);
    void help(Student& other);
    void exercise( );
private:
    int otherFunc( );
    ...
private:
    int score;
    int strength;
};
```

```
int main( ) {
    Student s1(0,10);
    Student s2(5,9);
    Course oop;
    s1.study(oop);
    s2.study(oop);
    s1.help(s2 );
    s1.otherFunc();
    return 0;
}
```

```
class Course {
public:
    int getDifficulty( )
        { return difficulty;}
private:
    int preiodCount( );
    int courseId;
    int difficulty;
};
```

```
class Student {
public:
    Student(int sc,int st);
    void study(Course &
course);
    void help(Student& other);
    void exercise();
private:
    int otherFunc();
    ...
private:
    int score;
    int strength;
};
```

```
//student.cpp
#include "student.h"
#include "course.h"
void Student:: study(Course & course) {
    score += 1;
    strength -= course.getDifficulty();
    otherFunc( );    //OK
    int n = course.periodCount( );
}
void Student:: help(Student& other){
    other.score += 2;
    strength -= 2;
    other.strength -= 4;
    other.exercise( );
    otherFunc( );    // OK
    other.otherFunc(); //OK
}
```

```
//main.cpp
#include "student.h"
void func(Student& other){
    other.score += 2;
    other.strength -= 4;
    other.exercise( );

    other.otherFunc();
}
int main() {
    // .....
    return 0;
}
```



- ◆ 封装：将实体的特征和相关信息，通过打包的过程，包装成一个整体
  - ✓ 对实体的内部：沟通不受影响
  - ✓ 对实体的外部：只能通过实体公开的特征进行沟通，不必了解实体的细节
  - ✓ C++中封装的实现手段---- class
- ◆ 信息隐蔽：在封装的过程中，使实体的部分信息对外部不可见，只公开部分必要信息
  - ✓ C++中信息隐蔽的实现手段---- 访问控制



封装与信息隐蔽是面向对象的基石，在程序设计中的作用包括：分离使用 and 实现（函数实现、数据组织、数据表示等）、分离接口与实现、软件复用

## //分离使用 and 实现

```
int MyClass::doSomething( List & list )
{
    list.push_back(1);
    list.push_back(2);
    list.push_back(3);
}
```

函数用到了一个List对象，只知道List提供了一个叫push\_back的操作，可以把一个元素放到集合末尾，不知道List内部是用数组、链表、还是平衡树实现的。调用者只关心List能做什么，不关心它怎么做的，List实现者可以自由更换底层数据结构，只要push\_back的行为接口不变，调用者一行代码都不用改。

- 调用者与类的内部实现的关系
- 隐藏 如何做到 的实现细节和复杂逻辑
- 受益方是调用者，关注调用者的体验，让调用者从复杂的实现细节中解脱，只需关心功能，不关心实现，使调用代码简单、干净
- 例如：油门踏板就是接口，发动机就是实现，分离之后，换发动机不影响踩油门的方式
- 黑盒复用，因为使用简单、依赖清晰，这个类，例如一个设计良好的List类，可以被不同的调用者复用
- 对应ADT的信息隐藏原则

## //分离接口与实现

```
class Game { //设计者定义的 Game 类 (接口契约)
public:
    virtual void init() = 0; // 初始化接口
    virtual void run() = 0; // 运行接口
    virtual void term() = 0; // 终止接口
    virtual ~Game() {}
};

class RpgGame : public Game { // 实现1: RPG游戏
    void init() override { /* 加载剧本和角色数据 */ }
    void run() override { /* 回合制战斗逻辑 */ }
    void term() override { /* 保存存档 */ }
};

int main() { //分离使用与实现
    Game& myGame = CreateRpgGame();
    game.init();
    game.run();
    game.term();
} //调用者代码极其简单、稳定
```

设计者/实现者考虑得更远，**建立一个更合理、更稳定、更具扩展性的接口契约**：将游戏的生命周期清晰地划分为三个阶段，符合大多数游戏的通用模型，是一个合理抽象；只要这个生命周期模型不变，这个三函数接口就可以非常稳定；这个接口允许存在无数种实现。

- **接口契约与实现该契约的具体实体的关系**
- **隐藏** 谁来履行 接口契约的具体身份和策略
- **受益方是设计者/实现者**，关注设计者/实现者的自由，让其能灵活变更、优化和替换内部实现，而不影响所有现有的使用者，**接口稳定，实现可变**
- 电源插座接口（两脚/三脚）是稳定标准，墙内的电路电线可以灵活变更
- 白盒复用，接口契约可被不同的实现者复用
- 对应ADT的以操作接口定义类型的原则



## ■ 行为的函数表示：

- ◆ 一般成员函数
- ◆ **常成员函数**
- ◆ 重载的成员函数
- ◆ 构造、析构、拷贝构造、赋值函数
- ◆ 自动转换函数
- ◆ 类方法

## 常成员函数格式：

<返回类型> <函数名> (<参数表>) **const**;  
非静态成员函数在声明和定义时，参数列表后面加上**const**限定符

```
class MyClass
{
    public: ...
    private:
        int func( ) const;
    private:
        int value;
};
```



- ◆常成员函数的含义
- ◆常成员函数的实现机制
- ◆常成员函数的访问
- ◆常成员函数的重载

```
class MyClass {  
public:  
    int myFunc( ) { value = value + 1; }  
    int func1( ) const { return value;}  
    int func2( ) const { return ++value;}  
private:  
    int value; // 非静态数据成员  
};
```

普通成员函数的this指针类型：

`T * const this`

常成员函数的this指针类型：

`const T * const this`

```
int main( ) {  
    MyClass obj;  
    obj.myFunc( ); //MyClass::myFunc(&obj)  
    obj.func1( );  //MyClass::func1(&obj)  
  
    const MyClass cObj;  
    cObj.myFunc( ); //MyClass::myFunc(&cObj)  
    cObj.func1( );  //MyClass::func1(&cObj)  
    return 0;  
};
```



# 常成员函数的含义和作用机制

- ◆常成员函数的含义
- ◆常成员函数的实现机制
- ◆常成员函数的访问
- ◆常成员函数的重载

```
void ff(MyClass & va) {  
    va.add( );  
    va.func( ); //--value  
    va.func(2);  
};
```

```
void gg(const MyClass & ca) {  
    ca.add( ); //wrong  
    ca.func( ); //const  
    ca.func(2); //wrong  
};
```

```
class MyClass {  
    public:  
        ...  
        void add( )    { ++value;}  
        int  func( )  
                { return --value; }  
        void func(int n)  
                { value = n; }  
        int  func( ) const  
                { return value;}  
    private:  
        int value;  
};
```





**问题1：** 什么时候定义常成员函数？

**问题2：** 不使用常成员函数可以吗？

**//当成员函数，不修改当前对象**

```
class MyClass {  
public:  
    int  getNum ( ) const  
        { return num; }  
    void setNum (int n)  
        { num = n; }  
private:  
    int num;  
};
```

**//反例：未声明为常成员函数**

```
class A {  
public:  
    void f( );  
    void g( );  
    //虽不修改当前对象，但未声明为常成员函数  
};  
void func(const A & a){  
    a.f( ); //wrong  
    a.g( ); //wrong  
}
```

**实例变(常)量:** 每个对象独立拥有的数据成员, C++用非静态数据成员表示

**实例方法:** 访问实例变(常)量的成员函数, C++用非静态成员函数表示

```
// student.h
class Student {
public:
    Student(int n);
    void FuncOne();
    int  FuncTwo(int param);
    //其它
private:
    const int id;
    int score;
};
```

```
//student.cpp
void Student:: FuncOne() {
    this->score = this->score + 5;
}
int  FuncTwo(int param){
    return this->score + param;
}
```

```
//main.cpp
int main() {
    Student st1,st2;

    st1.FuncOne(); // 通过对象访问, 3种形式
    st2.FuncTwo(8); // 通过对象访问, 3种形式

    return 0;
};
```



- ◆ 类变量：类具有的属性或应由类的所有对象共享的属性
  - ◆ C++ 中用静态数据成员表示
  - ◆ 类变(常)量的内存位置
  - ◆ 类变(常)量的初始化
- ◆ 类方法：类属的行为表示
  - ◆ C++ 中用静态成员函数表示
  - ◆ 不能直接访问任何非静态成员（数据成员和函数成员）
  - ◆ 没有隐含的this指针, 也不能带const修饰
  - ◆ 常用类名作限定（最好不用对象名）
    - 好： `ClassName::OneStaticFunction( );`
    - 不好： `objName.OneStaticFunction( );`

```
// example.h
class Example {
public:
    static int nextNum(Example& ex) {
        this->funcOne( ); //错误
        ++value;           //错误
        num += ex.value;   //正确
        return ++num;      //正确
    }
    static int subNum();
    Example(int n);
    void funcOne( );
private:
    static int num;
    int value;
};
```

```
// example.cpp
#include "example.h"
//静态成员在类外定义时不再指定static
//静态成员变量的定义
int Example::num = 1000;
//静态成员函数的定义
int Example::subNum() {
    return num = num - 1;
}
void Example::funcOne() {
    //略
}
int main( ) {
    int val = Example::subNum();
    ...
}
```



只用一副无大小王的扑克，扑克的花色(suit)分为 Spade、Heart、Diamond和 Club，每门花色的牌共13张，面值(rank)分别为 2、3、4、5、6、7、8、9、10、Jack、Queen、King和Ace，每张扑克牌应包含如下信息：唯一的ID号(1-52)、花色、面值、背面图案的编号、扑克牌的宽度、扑克牌的高度、扑克牌的左顶点坐标等。

扑克牌的操作有：

1. 存取背面图案；
2. 判断与另一张扑克牌是相同花色吗？
3. 判断与另一张扑克牌是相同面值吗？
4. 判断扑克牌是给指定的花色吗？
5. 判断扑克牌是给指定的面值吗？
6. 设置扑克牌坐标，
7. 取得扑克牌的右下角坐标等。



## 类变(常)量和类方法 (例)

扑克牌的操作有：

1. 存取背面图案；
2. 判断与另一张扑克牌是相同花色吗？
3. 判断与另一张扑克牌是相同面值吗？
4. 判断扑克牌是给指定的花色吗？
5. 判断扑克牌是给指定的面值吗？
6. 设置扑克牌坐标，
7. 取得扑克牌的右下角坐标等。

扑克牌的数据有：

背面图案的编号、

唯一的ID号(1-52)、

花色、

面值、

扑克牌的宽度、

扑克牌的高度、

扑克牌的左顶点坐标等

```
class Card {  
public:
```

```
    int  getBackImageld( ) const;  
    void setBackImageld(int id ) ;  
    bool isSameSuit(const Card& c) const;  
    bool isSameRank(const Card& c) const;  
    bool isSuit(Suit s) const;  
    bool isRank(Rank r) const;  
    void setPosition(int x,int y);  
    Point getPositonRB ( ) const;
```

```
private:
```

```
    int  backImageld;  
    int  ID;  
    Suit suit;  Rank rank;  
    int  width , height;  
    int  xPos,yPos;
```

```
};
```



## Card类的定义和实现 (例续)

```
class Card {
public:
    int getBackImageld( ) const;
    void setBackImageld(int id ) ;
    ...
private:
    int backImageld;
    int ID;
    Suit suit; Rank rank;
    int width , height;
    int xPos,yPos;
};
```

```
class Card {
public:
    static int getBackImageld( ) const
        { return backImageld; }
    static void setBackImageld(int id )
        { backImageld = id; }
    ...
private:
    static int backImageld;
    const int ID;
    const Suit suit; const Rank rank;
    int width , height;
    int xPos, yPos;
};
```

在某个cpp文件中定义并初始化backImageld:  
`int Card::backImageld = 2;`



## Card类的定义和实现 (例续)

```
enum class Suit { CLUB = 0, DIAMOND, HEART, SPADE };  
enum class Rank  
    { TWO = 2, THREE, FOUR, FIVE, SIX, SEVEN, EIGHT, NINE, T, J, Q, K, ACE = 1 };
```

```
class Card {  
public:  
    Card(int id):  
ID(id),suit((Suit)(id/13)),rank(Rank(id%13+1))  
    { }  
    const char * suitName( ) const;  
    const char * ranktName( ) const;  
    ...  
private:  
    const int ID;  
    const Suit suit;  const Rank rank;  
    ...  
};
```

```
// card.cpp  
const char * Card::suitName( ) const {  
    static const char * names[ ] =  
        { "C","D","H","S"};  
    return names[(int)suit];  
}  
const char * Card::ranktName() const {  
    static const char * names[ ] =  
    { "", "A", "2", "3", "4", "5", "6", "7", "8", "9", "T", "J", "Q", "K" };  
    return names[(int)rank];  
}
```





## Card类的定义和实现（例续）

```
class Card {  
public:  
  
    ...  
    bool isSameSuit(const Card& c) const      { return suit == c.suit; }  
    bool isSameRank(const Card& c) const      { return rank == c.rank; }  
    bool isSuit(Suit s) const                 { return s==suit; }  
    bool isRank(Rank r) const                 { return r==rank; }  
    void setPosition(int x,int y)              { xPos=x; yPos=y; }  
    Point getPositonRB ( ) const               { return  Point(xPos+width,yPos+height); }  
  
private:  
  
    ...  
    const int  ID;  
    const Suit suit;  const Rank rank;  
    int width , height;  
    int xPos,yPos;  
  
};
```



# 本章结束